



# **AgroSense\_Door & Window Sensor LoRaWAN® Manual V1.0**

Author: Yuki

Time: 2026.06.08

# Contents

|                                    |          |
|------------------------------------|----------|
| <b>1 Product Description .....</b> | <b>1</b> |
| 1.1 Introduction .....             | 1        |
| 1.2 Feature .....                  | 2        |
| 1.3 Parameter .....                | 3        |
| <b>2 Technical route .....</b>     | <b>4</b> |
| 2.1 System Framework .....         | 4        |
| 2.2 Regional frequency band .....  | 5        |
| <b>3 Usage .....</b>               | <b>6</b> |
| 3.1 TTN and ThingSpeak .....       | 6        |
| 3.2 Datacake .....                 | 16       |

# 1 Product Description

## 1.1 Introduction

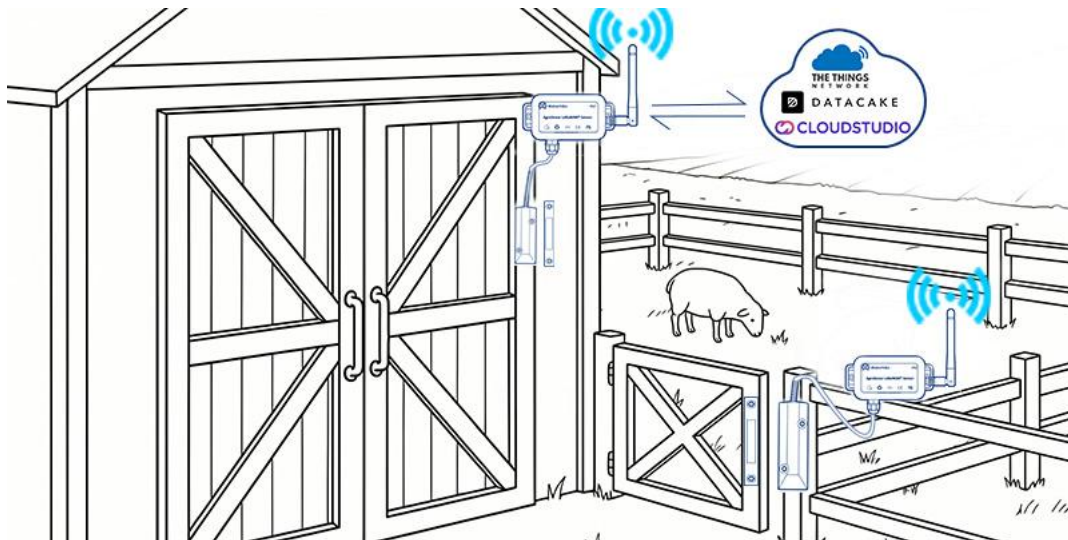
The AgroSense Magnetic Contact sensor is used for window/door monitoring over the LoRaWAN® network. It detects the window/door status, and with cloud service such as DataCake, alarm you via email/whatsapp/MQTT, etc. Can be used for warehouse monitoring and Livestock management on farm.

This product supports common 2 data modes:

- Data Uplink: It uplink the magnetic contacts status(contacted or not, time& count of door openings, etc ) every 12 hours, and instant uplink as status changes(from contacted to NOT, and wise visa).
- Data Downlink: Users can send downlink commands from cloud server, to change the update interval to data uplink(12 hours by default)

This Module supports global frequency bands (EU868/US915) and works seamlessly with platforms such as TTN, Datacake, and CloudStudio to enable data upload and remote control.

Besides, it has Local storage for max 3,300+ records, ensures critical data is retained during connectivity gaps.





## 1.2 Feature

- Includes a **high precision** sensor.
- Compatible with Worldwide **LoRaWAN® Networks**: Support the universal frequency bands EU868/ US915.
- LoRaWAN version: LoRaWAN Specification 1.0.3.
- **Long Range**: Up to 2 kilometers in the city, up to 10 kilometers in the wilderness, receive sensitivity -137dBm , transmit power up to 21dBm.
- Waterproof IP67, 5~8 years working time with default battery/setting.
- **Data encryption**: Provide end-to-end secure communication, including device authentication and network data encryption, to ensure the security of data transmission and prevent data theft and malicious attacks.
- **High stability and reliability**: good stability in noisy environments, able to penetrate buildings and obstacles, so it can maintain good communication quality in urban and suburban environments.
- Suitable for **Harsh Environments**: Can work normally under the temperature of -40℃ ~ 85℃, IP68 waterproof, suitable for outdoor use in harsh conditions, high UV, dusty, heavy rain and other bad weather.
- Monitor data and upload **real-time** data regularly.
- Modify the product parameters through **AT commands**.

## 1.3 Parameter

### 1. General Parameters

|                    |          |
|--------------------|----------|
| Product Model      | AGLWDW   |
| Detection Distance | 20–40 mm |
| Cable Length       | 1.5 m    |

### 2. Wireless Parameters

|                               |                                   |
|-------------------------------|-----------------------------------|
| Communication Protocol        | Standard LoRaWAN® protocol V1.0.3 |
| Network Access/Operating Mode | OTAA Class A                      |
| MAX Transmit Power            | 22dBm                             |
| Receiver Sensitivity          | -137dBm/125kHz SF=12              |
| Frequency Band                | EU868/US915                       |

### 3. Physical Parameters

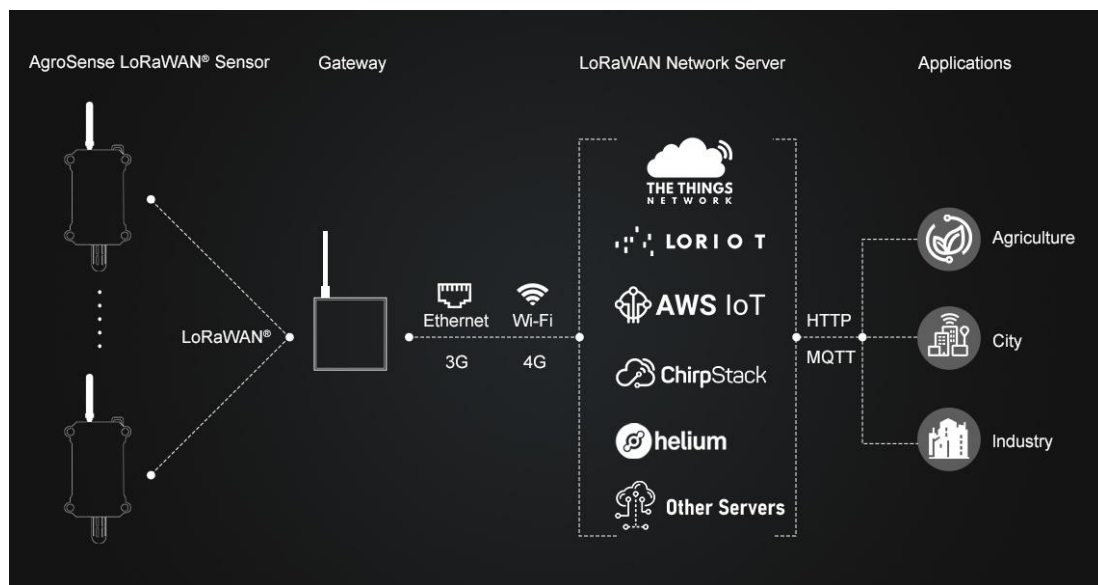
|                       |                             |
|-----------------------|-----------------------------|
| Power Supply          | 1 x 17450 3V Lion batteries |
| Operating Temperature | -40°C ~85°C                 |
| Protection Class      | IP67                        |
| Main Unit             | 101 × 67 × 31 mm            |
| Magnetic Contact      | 31 × 15 × 8 mm              |
| Mounting              | Wall Mounting               |

## 2 Technical route

### 2.1 System Framework

AgroSense Door & Window Sensor uses LoRaWAN technology, and its network architecture includes four parts: End Nodes, Concentrator/Gateway, Network Server and Application Server.

|                      |                                                                                                             |
|----------------------|-------------------------------------------------------------------------------------------------------------|
| End Nodes            | It is responsible for collecting sensing data and then transmitting it to Gateway via the LoRaMAC protocol. |
| Concentrator/Gateway | It is mainly responsible for transmitting node data to the server.                                          |
| Network Server       | Organize the data into JSON packets and decode them.                                                        |
| Application Server   | Display the data.                                                                                           |



The steps to achieve the detection of leak is:

1. Collect door-opening alerts from sensors, and send the data from End Node to Gateway.
2. The Gateway packages node data and transmits it to the Network Server.
3. The Network Server decodes the data and sends it to the Applications.
4. Finally, user can monitor the data in the APP.

## 2.2 Regional frequency band

At the present moment, our product solely accommodates compatibility with the US915 and EU868.

| area        | frequency band | center frequency |
|-------------|----------------|------------------|
| China       | 470-510MHz     | CN486MHz         |
| America     | 902-928MHz     | US915MHz         |
| Europe      | 863-870MHz     | EU868MHz         |
| Korea       | 920-923MHz     | KR922MHz         |
| Australia   | 915-928MHz     | AU923MHz         |
| New Zealand | 921-928MHz     | NZ922MHz         |
| Asia        | 920-923MHz     | AS923MHz         |

## 3 Usage

We use The Things Network or Datacake as our Network Server, we need to configuration the country/ area frequency, inputting DEV EUI/ APP EUI/ APP Key, decodes, and connect to ThingSpeak or Datacake.

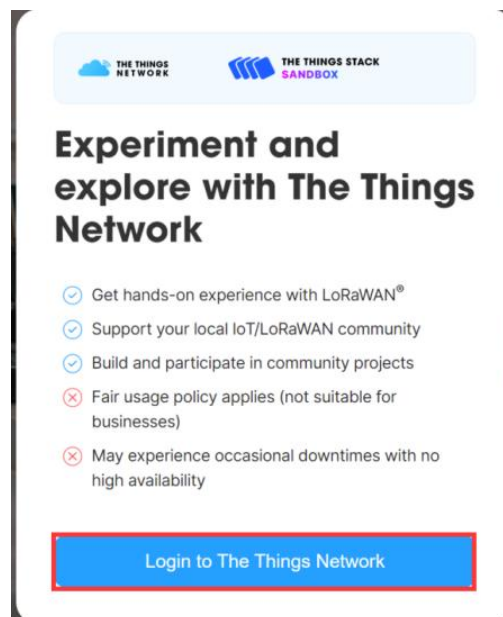
|         |                                                                       |
|---------|-----------------------------------------------------------------------|
| DEV EUI | Unique identification of device, authorized by IEEE                   |
| APP EUI | Unique identification of application                                  |
| APP Key | One of the join network parameters on OTAA mode, calculated by DE EUI |

- End Nodes and Gateway: AgroSense Door & Window Sensor.(The AgroSense series is applicable)
- Network Server: The Things Network. ( Loriot, AWS IoT, ChirpStack, ect)
- Application Server: ThingSpeak.(Datacake, Blockbax, akenza, ect)

### 3.1 TTN and ThingSpeak

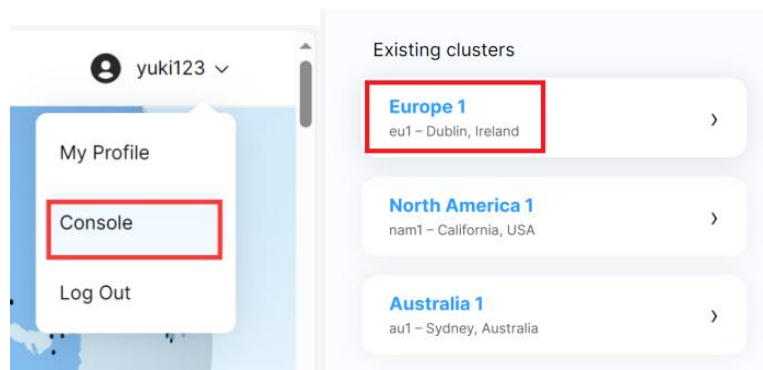
#### 3.1.1 Network Server configuration

- Open The Things Network in your browser and login it. (Or register an account)

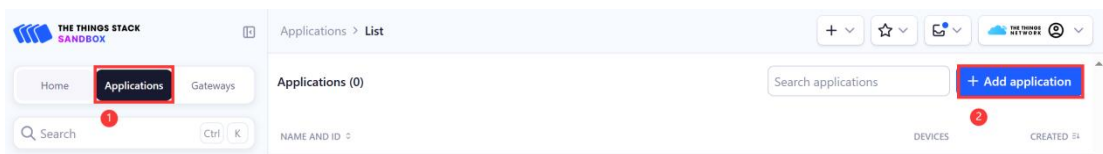




- Click “Console” and select clusters. (we take the European region for example.)



- Click “Go to applications” --> “+ Create application”.



- Write the Application ID and click “Create application”.

Application ID \*

agrosense-sensor

Application name

My new application

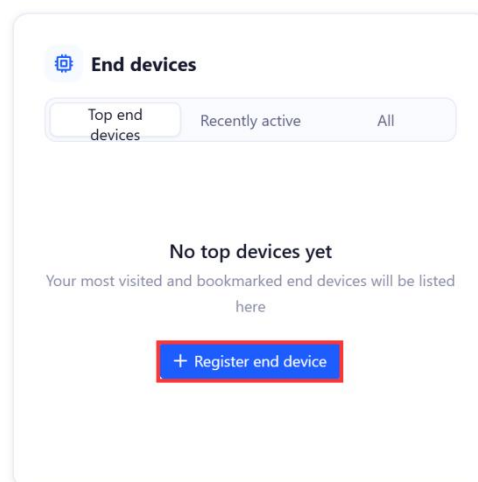
Description

Description for my new application

Optional application description; can also be used to save notes about the :

Create application

- Click “+ Register and device”.



- Following the steps, and input the DEV EUI/ APP EUI/ APP Key (notice: JoinEUI=APP EUI) and subsequently click on "Register end device" to complete the registration process.

**End device type**

Input method ⓘ

☐ Select the end device in the LoRaWAN Device Repository

☒ Enter end device specifics manually 1

**Frequency plan** ⓘ \*

Europe 863-870 MHz (SF9 for RX2 - recommended) | v

**LoRaWAN version** ⓘ \*

LoRaWAN Specification 1.0.3 | v

**Regional Parameters version** ⓘ \*

RP001 Regional Parameters 1.0.3 revision A | v 2

[Show advanced activation, LoRaWAN class and cluster settings](#)

**Provisioning information**

JoinEUI ⓘ \*

48 FF 00 00 00 00 01 65

Confirm

3 Continue, please enter the JoinEUI 4 and device so we can determine onboarding options

**Provisioning information**

**JoinEUI** ⓘ \*

48 FF 00 00 00 00 01 65 Reset

This end device can be registered on the network

**DevEUI** ⓘ \*

48 E6 63 FF FE 30 01 65 Generate 0/50 used

**AppKey** ⓘ \*

4A 35 62 6B 95 AB 5B 4D 3F 3B DE 12 71 B1 6F 2A Generate

**End device ID** ⓘ \*

eui-48e663fffe300165

This value is automatically prefilled using the DevEUI

**After registration**

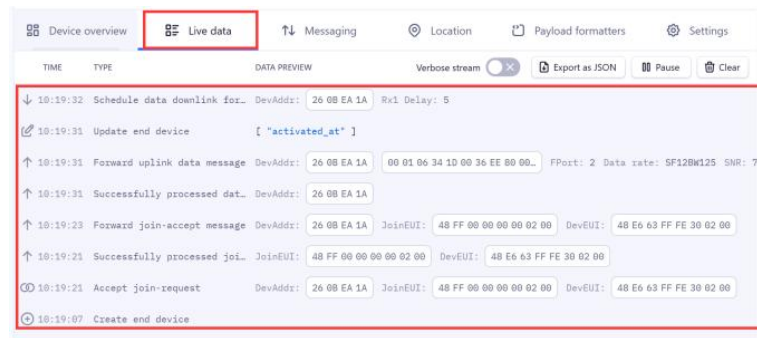
☒ View registered end device

☐ Register another end device of this type

Register end device

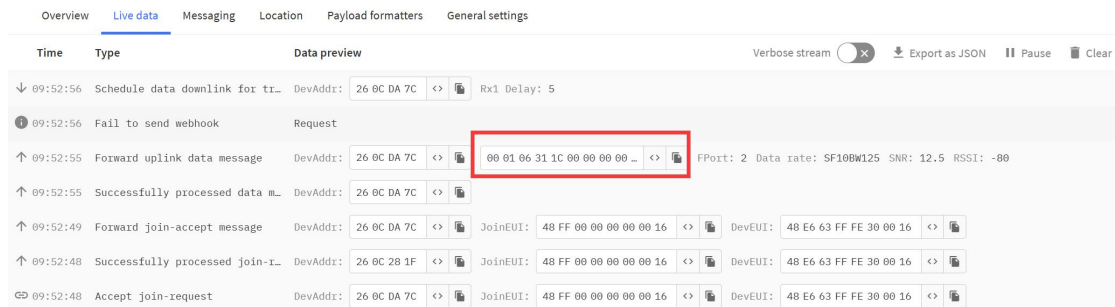


- Plug the battery and press RES button, you can see the device is connected successfully in the TTN.



### 3.1.2 Decoder

- Now, we need to decode the data.



| Data length | Data description                           | Value range | Explanation                                                                                                                                                                                                                                                                                                           |
|-------------|--------------------------------------------|-------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| byte 0      | Data packet sequence number<br>high 8 bits | 0-0xFFFF    | Counting starts from 0 and increments, resetting back to 0 after reaching 65535                                                                                                                                                                                                                                       |
| byte 1      | Data packet sequence number<br>low 8 bits  |             |                                                                                                                                                                                                                                                                                                                       |
| byte 2      | Battery voltage                            |             | The value is obtained by amplifying the data by 10 times, and the actual value needs to be divided by 10 to convert to the actual battery voltage.<br>The purpose of multiplying by 10 is to retain one decimal place of the voltage value. For example, if the value is 0x21 = 33, then the battery voltage is 3.3V. |
| byte 3      | Door open state                            | 0/1         | Current door status. 0 = Door closed, 1 = Door open.                                                                                                                                                                                                                                                                  |
| byte 4      | Door open count bits 8 to 15               |             | Door open counter. It records the total number of times the door has been opened. For example, if the value is 0x002C (44), then the door has been opened 44 times.                                                                                                                                                   |
| byte 5      | Door open count bits 0 to 7                |             |                                                                                                                                                                                                                                                                                                                       |
| byte 6      | Last door open duration bits 24 to 31      |             | Duration of the most recent door opening event during the time interval.<br>The unit is seconds. For example, if the value is 0x0000003C (60), then the last door open duration was 60 seconds.                                                                                                                       |
| byte 7      | Last door open duration bits 16 to 23      |             |                                                                                                                                                                                                                                                                                                                       |

### AgroSense Door & Window Sensor LoRaWAN®

|                |                                                    |   |                                                                                                                                                                                                                                                            |
|----------------|----------------------------------------------------|---|------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>byte 8</b>  | Last door open duration bits 8 to 15               |   |                                                                                                                                                                                                                                                            |
| <b>byte 9</b>  | Last door open duration bits 0 to 7                |   |                                                                                                                                                                                                                                                            |
| <b>byte 10</b> | Total accumulated door open duration bits 24 to 31 |   | Total accumulated door-open time during the time interval. The unit is seconds. For example, if the value is 0x0000012C (300), then the accumulated open duration is 300 seconds.                                                                          |
| <b>byte 11</b> | Total accumulated door open duration bits 16 to 23 |   |                                                                                                                                                                                                                                                            |
| <b>byte 12</b> | Total accumulated door open duration bits 8 to 15  |   |                                                                                                                                                                                                                                                            |
| <b>byte 13</b> | Total accumulated door open duration bits 0 to 7   |   |                                                                                                                                                                                                                                                            |
| <b>byte 14</b> | Data transmission interval bits 24 to 31           |   | Uplink data transmission interval. The unit is seconds. For example, if the value is 0x0000012C (300), then the transmission interval is 300 seconds (5 minutes).                                                                                          |
| <b>byte 15</b> | Data transmission interval bits 16 to 23           |   |                                                                                                                                                                                                                                                            |
| <b>byte 16</b> | Data transmission interval bits 8 to 15            |   |                                                                                                                                                                                                                                                            |
| <b>byte 17</b> | Data transmission interval bits 0 to 7             |   |                                                                                                                                                                                                                                                            |
| <b>byte 18</b> | Timestamp bits 24 to 31                            |   | These 4 bytes form a 32-bit Unix timestamp. It represents the number of seconds elapsed since 1970-01-01 00:00:00 (UTC) when this data record was generated. After combining the 4 bytes, the value can be converted into the corresponding date and time. |
| <b>byte 19</b> | Timestamp bits 16 to 23                            |   |                                                                                                                                                                                                                                                            |
| <b>byte 20</b> | Timestamp bits 8 to 15                             |   |                                                                                                                                                                                                                                                            |
| <b>byte 21</b> | Timestamp bits 0 to 7                              |   |                                                                                                                                                                                                                                                            |
| <b>byte 22</b> | System reserved bits                               | 0 | Always 0 (and can be ignored)                                                                                                                                                                                                                              |

Example: 0x00, 0x27, 0x21, 0x01, 0x00, 0x02, 0x00, 0x00, 0x01, 0xFE, 0x00, 0x00, 0x02, 0xA8, 0x00 0x36 0xEE 0x80 0x69 0xCD 0xCD 0xED 0x00

Data parsing:

Battery voltage is 3.3 V.

Door open count is 1.(The door is currently open.)

Last door open duration is 510s.

Total accumulated door open duration is 600s.

Data transmission interval value is 3600s.

Timestamp value is 1775095277 ( 2026-04-02 02:01:17 ).

- Know how to decode it after, we need to write it in code. (you can check it out on [Github](#))

```
function decodeUplink(input) {
    var num = input.bytes[0] * 256 + input.bytes[1]
```

```

var bat = input.bytes[2] / 10.0
var Door_open_flag = input.bytes[3]
var Door_open_cnt = input.bytes[4] * 256 + input.bytes[5]
var Door_open_last_time = input.bytes[6] * 16777216 + input.bytes[7] * 65536 + input.bytes[8] * 256 +
input.bytes[9]
var Door_open_all_time = input.bytes[10] * 16777216 + input.bytes[11] * 65536 + input.bytes[12] * 256 +
input.bytes[13]
var interval = (input.bytes[14] * 16777216 + input.bytes[15] * 65536 + input.bytes[16] * 256 +
input.bytes[17]) / 1000 //s
var time = (input.bytes[18]*16777216 + input.bytes[19]*65536 + input.bytes[20]*256 + input.bytes[21])

var timeStr = null;

var d = new Date(time * 1000)

timeStr =
    d.getUTCFullYear() + "-" +
    String(d.getUTCMonth() + 1).padStart(2, "0") + "-" +
    String(d.getUTCDate()).padStart(2, "0") + " " +
    String(d.getUTCHours()).padStart(2, "0") + ":" +
    String(d.getUTCMinutes()).padStart(2, "0") + ":" +
    String(d.getUTCSeconds()).padStart(2, "0");
return {
    data: {
        field0: num,
        field1: bat,
        field2: Door_open_flag, //Normal is 0,Door_open is 1.
        field3: Door_open_cnt,
        field4: Door_open_last_time,
        field5: Door_open_all_time,
        field6: interval,
        device_time: timeStr,
        device_time_unix: time
    },
};
}

```

- Select “Payload formatters” and follow the steps.

Device overview Live data Messaging Location **Payload formatters**

Uplink Downlink

Setup

Formatter type\*  
Custom Javascript formatter

Formatter code\*

```
function decodeuplink(input) {
  var num = input.bytes[0] * 256 + input.bytes[1];
  var bat = input.bytes[2] / 1000;
  var Door_open_flag = input.bytes[3];
  var Door_open_cnt = input.bytes[4] * 256 + input.bytes[5];
  var Door_open_last_time = input.bytes[6] * 16777216 + input.bytes[7] * 65536 + input.bytes[8] * 256 + input.bytes[9];
  var Door_open_all_time = input.bytes[10] * 16777216 + input.bytes[11] * 65536 + input.bytes[12] * 256 + input.bytes[13];
  var interval = input.bytes[14] * 16777216 + input.bytes[15] * 65536 + input.bytes[16] * 256 + input.bytes[17];

  var time = null;
  var timeStr = null;

  if (input.bytes.length >= 18) {
    time = (input.bytes[18] * 16777216 + input.bytes[19] * 65536 + input.bytes[20] * 256 + input.bytes[21]);
    var d = new Date(time * 1000);
    timeStr =
      d.getUTCFullYear() + "-" +
      String(d.getUTCMonth() + 1).padStart(2, "0") + "-" +
      String(d.getUTCDate()).padStart(2, "0") + "T" +
      String(d.getUTCHours()).padStart(2, "0") + ":" +
      String(d.getUTCMinutes()).padStart(2, "0") + ":" +
      String(d.getUTCSeconds()).padStart(2, "0") + "Z";
  }

  return {
    bat: bat,
    Door_open_flag: Door_open_flag,
    Door_open_cnt: Door_open_cnt,
    Door_open_last_time: Door_open_last_time,
    Door_open_all_time: Door_open_all_time,
    interval: interval,
    time: time,
    timeStr: timeStr
  };
}
```

Save changes

### 3.1.3 Application Server configuration

In the Application Server configuration, we need to create ThingSpeak channel and get Channel ID and API Key, this is the key to our connection to TTN.

- Login to the ThingSpeak. (Or register an account)

MathWorks®

Email

No account? [Create one!](#)

By signing in, you agree to our [privacy policy](#).

Next

- Click “New Channel”, fill in the Channel name and field names and click “Save Channel”.

ThingSpeak™

My Channels

New Channel

Name: AgroSense\_Magnetic Contact Switch

Description:

Field 1: bat ✓

Field 2: Door\_open\_flag ✓

Field 3: Door\_open\_cnt ✓

Field 4: Door\_open\_last\_time ✓

Field 5: Door\_open\_all\_time ✓

Field 6: interval ✓

Save Channel

- After successful creation, copy the Channel ID and API Key.

## AgroSense Soil Moisture Sensor

Channel ID: 2599652  
 Author: mwa0000034232775  
 Access: Private

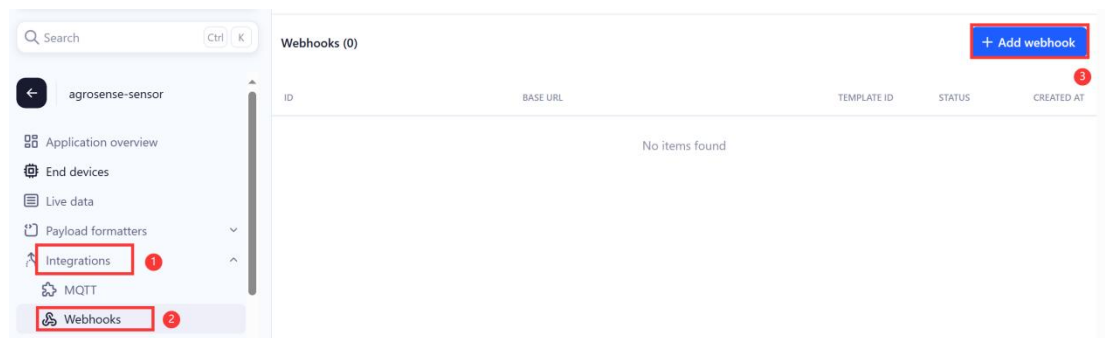
Private View Public View Channel Settings Sharing API Keys

### Write API Key

Key N9IBFTBI3J36T779

### 3.1.4 Connect the Network Server and Application Server

- In the TTN, click “integrations” --> “Webhooks” --> “+ Add webhook”.



- Select “ThingSpeak”, Fill in the Webhook ID and paste the Channel ID and API Key, click “Create ThingSpeak Webhook”.



**ThingSpeak**

Send data to ThingSpeak channel

Webhook ID\*

Channel ID\*

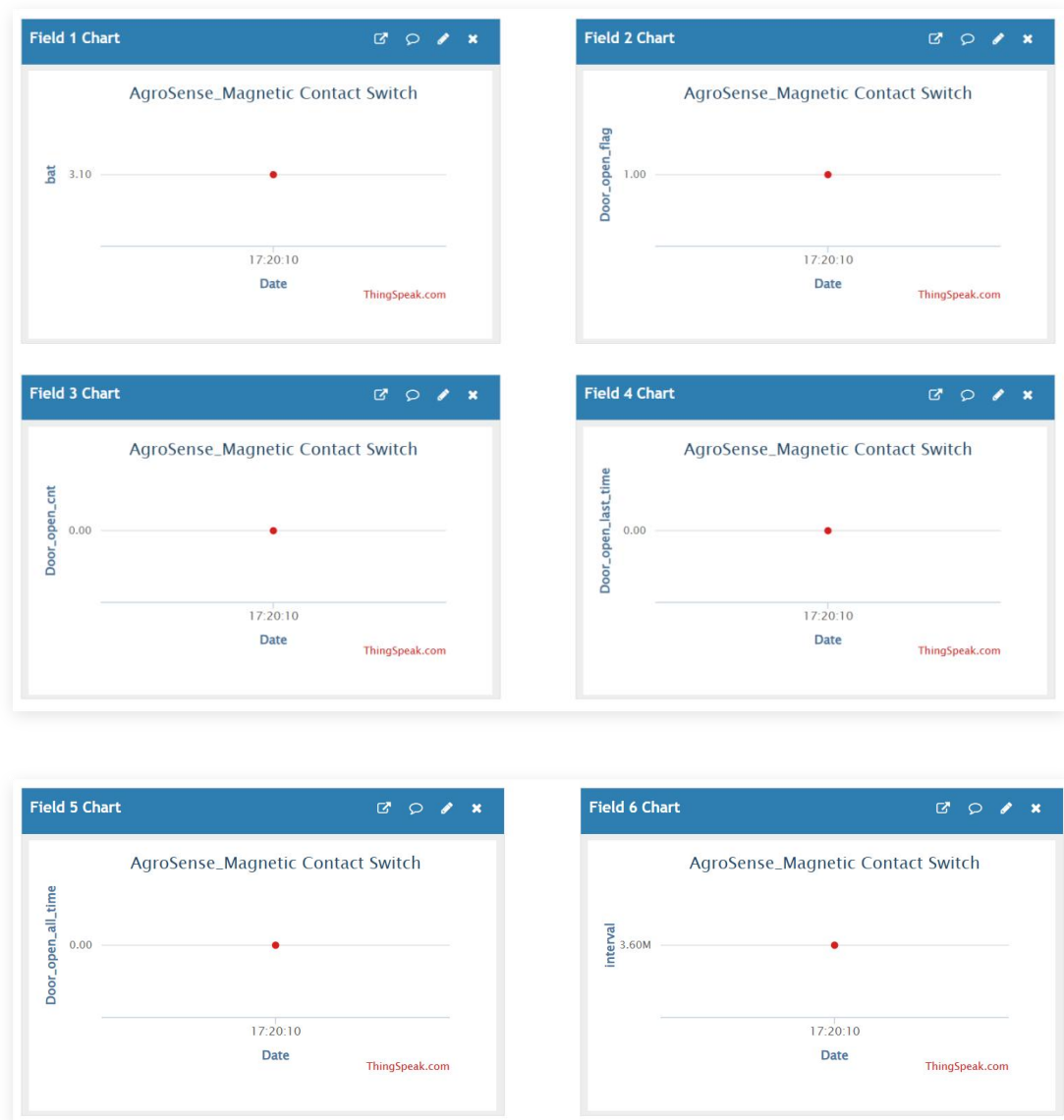
ThingSpeak Channel ID

API Key\*

ThingSpeak Write API Key

**Create ThingSpeak webhook**

- Press RST button, wait about a minute, you will successfully see the data in ThingSpeak.



| Leakage situation          | Reporting of data                                                                |
|----------------------------|----------------------------------------------------------------------------------|
| Normal state (Door close ) | Data reported every 12 hours.                                                    |
| Occasional open            | Report once when opening is detected, and report again when closing is detected. |
| Persistent open            | Reported every 10 minute, up to 5 consecutive times.                             |

### 3.1.5 Downlink

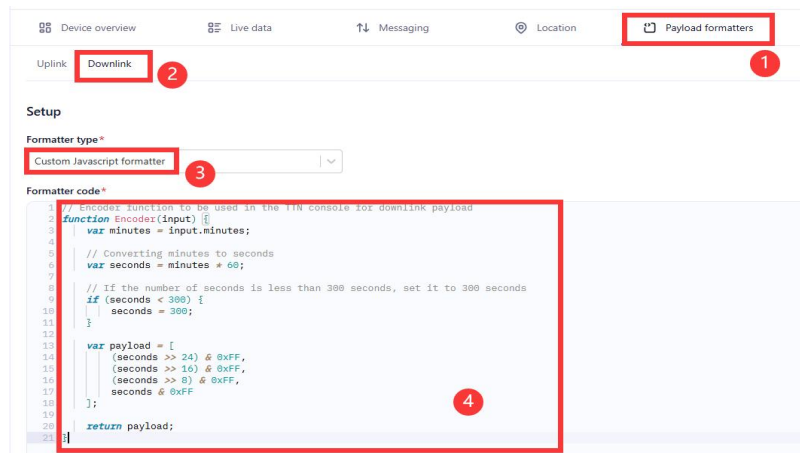
- Modification time interval (Fport1)

Modify the time interval for uploading data, the default is one hour.

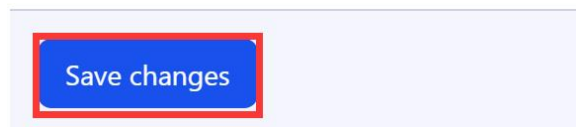
1、 If you need to change time Interval (Default 12 hours), you can click “Payload formatters-->Downlink” and follow the steps.



Formatter code you can find in [Github](#).



2、Click “Save changes”.



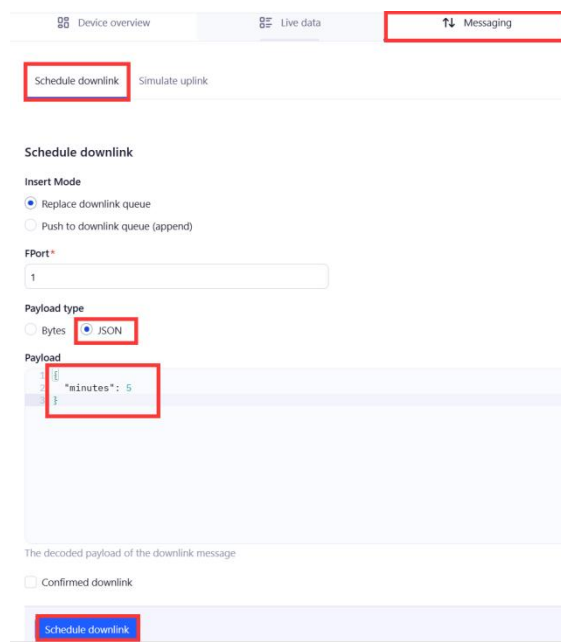
3、Click “Messaging-->Schedule downlink”.

**Note:** you must use this format:

```

{
  "minutes": 60
}

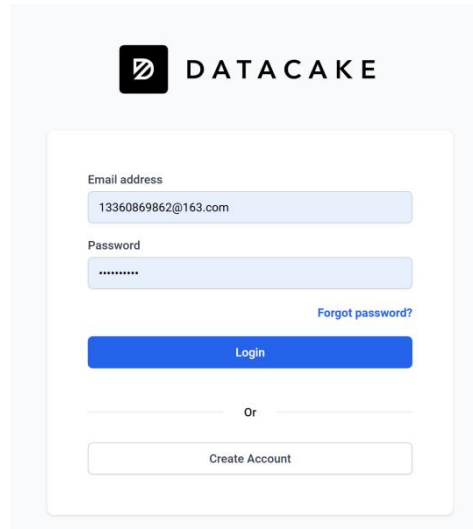
```



4、The modified interval will be updated after the next data upload.

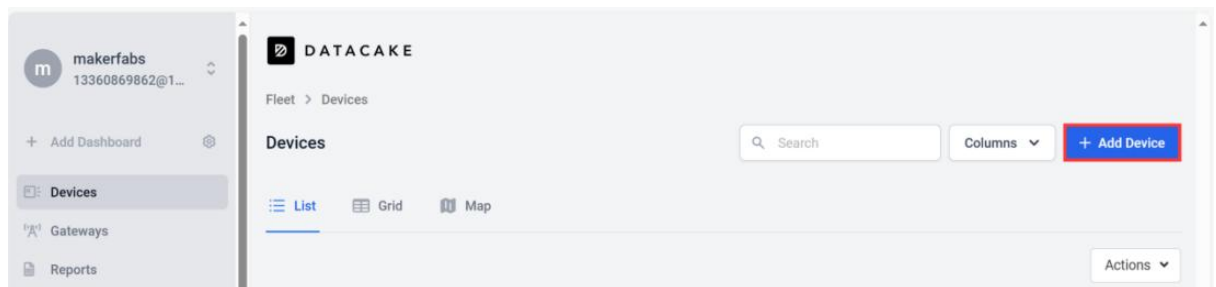
## 3.2 Datacake

### 1、Login datacake or Create Account

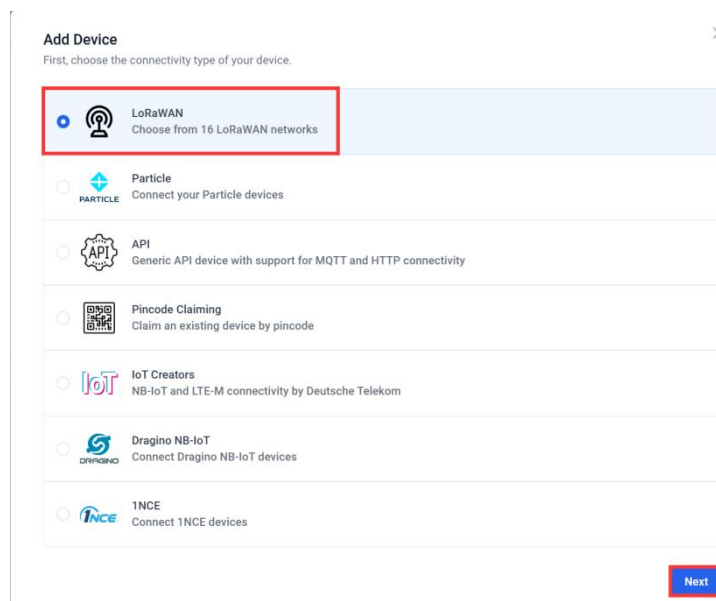


The image shows the Datacake login and registration interface. At the top is the Datacake logo. Below it is a form with two input fields: 'Email address' (containing '13360869862@163.com') and 'Password' (containing '\*\*\*\*\*'). There is a 'Forgot password?' link next to the password field. Below the password field is a blue 'Login' button. Underneath the login button is an 'Or' separator, followed by a white 'Create Account' button.

### 2、Click “Add Device”



### 3、Select LoRaWAN and click “Next”



The image shows the 'Add Device' dialog box. It has a title 'Add Device' and a subtitle 'First, choose the connectivity type of your device.' Below this is a list of connectivity options, each with a radio button and an icon. The first option, 'LoRaWAN', is selected and highlighted with a red box. The other options are: 'Particle' (with a blue diamond icon), 'API' (with a gear icon), 'Pincode Claiming' (with a QR code icon), 'IoT Creators' (with a colorful 'IoT' logo), 'Dragino NB-IoT' (with a blue 'S' icon), and '1NCE' (with a blue '1NCE' logo). At the bottom right of the dialog is a blue 'Next' button, also highlighted with a red box.

4、Select a Product based on your needs, take "Create new empty product" as an example.

5、Select "Datacake LNS"

6、Enter DEVEUI、APPEUI、APPKEY、FREQUENCY(take 915 for example) and DEVICE CLASS.

7、Choose the type according to your needs, and click “Add 1 device”.

8、Click to go to the device you just added.

| DEVICE                                        | PRIMARY | SECONDARY | DEVICE SIGNAL | DEVICE BATTERY |
|-----------------------------------------------|---------|-----------|---------------|----------------|
| AgroSense_Air Temperature and Humidity Sensor | 40.2    | 25        | -48           | 2.5            |
| AgroSense-carbon dioxide (CO2) sensor         | 0       | N/A       | -86           | 3.3            |
| AgroSense sensor                              | N/A     | N/A       | N/A           | N/A            |

9、Click “Configuration”, enter Decoder and click “Save”.(You can check it out on [Guihub](#))

```

function decode(payload, port) {
    var input = {
        bytes: payload
    };
    // var hex = input.bytes[0] * 256 + input.bytes[1];
    // var bat = input.bytes[2] / 1000;
    // var water_leak_flag = input.bytes[3];
    // var water_leak_pos = input.bytes[4] * 256 + input.bytes[5];
    // var water_leak_time = input.bytes[6] * 3677220 + input.bytes[7] * 4096 + input.bytes[8] * 256 + input.bytes[9];

    var decoded = {
        bat: bat,
        water_leak_flag: water_leak_flag, //normal is 0, inverse is 1
        water_leak_pos: water_leak_pos,
        water_leak_time: water_leak_time
    };

    // Test for LoRa properties in normalizedPayload
    try {
        console.log('normalizedPayload', normalizedPayload); // log to check normalizedPayload structure
        decoded.data_payload = normalizedPayload;
        decoded.data_payload.getPayloads().length > 0 && normalizedPayload.getPayloads()[0].payload() || 0;
        decoded.data_payload.getPayloads().length > 0 && normalizedPayload.getPayloads()[0].payload() || 0;
        decoded.data_payload.getPayloads().length > 0 && normalizedPayload.getPayloads()[0].payload() || 0;
        console.log('Error occurred while decoding LoRa properties: ' + error);
    } catch (error) {
        console.log('Error occurred while decoding LoRa properties: ' + error);
    }
    return {
        payload: "bat", value: decoded.bat,
        payload: "water_leak_flag", value: decoded.water_leak_flag,
        payload: "water_leak_pos", value: decoded.water_leak_pos,
        payload: "water_leak_time", value: decoded.water_leak_time
    };
}
    
```

```
function Decoder(payload, port) {

    var input = {

        bytes: payload

    };

    var num = input.bytes[0] * 256 + input.bytes[1]

    var bat = input.bytes[2] / 10.0

    var Door_open_flag = input.bytes[3]

    var Door_open_cnt = input.bytes[4] * 256 + input.bytes[5]

    var Door_open_last_time = input.bytes[6] * 16777216 + input.bytes[7] * 65536 + input.bytes[8] * 256 +
input.bytes[9] //s

    var Door_open_all_time = input.bytes[10] * 16777216 + input.bytes[11] * 65536 + input.bytes[12] * 256 +
input.bytes[13] //s

    var interval = (input.bytes[14] * 16777216 + input.bytes[15] * 65536 + input.bytes[16] * 256 +
input.bytes[17]) / 1000 //s

    var time = (input.bytes[18]*16777216 + input.bytes[19]*65536 + input.bytes[20]*256 + input.bytes[21])

    var decoded =

    {

        NUM:num,

        BAT:bat,

        DOOR_OPEN_FLAG:Door_open_flag,

        DOOR_OPEN_CNT:Door_open_cnt,

        DOOR_OPEN_LAST_TIME:Door_open_last_time,

        DOOR_OPEN_ALL_TIME:Door_open_all_time,

        INTERVAL:interval,

        time: time

    };

    // Test for LoRa properties in normalizedPayload

    try {

        if (normalizedPayload.gateways && normalizedPayload.gateways.length > 0) {

            decoded.LORA_RSSI = normalizedPayload.gateways[0].rssi || 0;

        }

    }

}
```

```

        decoded.LORA_SNR = normalizedPayload.gateways[0].snr || 0;

    } else {

        decoded.LORA_RSSI = 0;

        decoded.LORA_SNR = 0;

    }

    decoded.LORA_DATARATE = normalizedPayload.spreading_factor

        || normalizedPayload.data_rate

        || (normalizedPayload.networks    &&    normalizedPayload.networks.lora    &&
normalizedPayload.networks.lora.dr)

        || "unknown";

    } catch (error) {

        console.log('LoRa property parsing error:', error);

        decoded.LORA_RSSI = 0;

        decoded.LORA_SNR = 0;

        decoded.LORA_DATARATE = "unknown";

    }

    return [

        { field: "BAT", value: decoded.BAT, timestamp: time },

        { field: "DOOR_OPEN_FLAG", value: decoded.DOOR_OPEN_FLAG, timestamp: time },

        { field: "DOOR_OPEN_CNT", value: decoded.DOOR_OPEN_CNT, timestamp: time },

        { field: "DOOR_OPEN_LAST_TIME", value: decoded.DOOR_OPEN_LAST_TIME, timestamp: time },

        { field: "DOOR_OPEN_ALL_TIME", value: decoded.DOOR_OPEN_ALL_TIME, timestamp: time },

        { field: "INTERVAL", value: decoded.INTERVAL, timestamp: time },

        { field: "LORA_RSSI", value: decoded.LORA_RSSI, timestamp: time },

        { field: "LORA_SNR", value: decoded.LORA_SNR, timestamp: time },

        { field: "LORA_DATARATE", value: decoded.LORA_DATARATE, timestamp: time }

    ];

```

}

10、 Follow the steps to add a field.

Fields

+ Add Field

Fields describe the data the device will store.

Add Field

Fields define the schema of the data the device stores.

Type

Numeric

Name

bat

Identifier

BAT

The field identifier is a unique string that can consist of uppercase letters, numbers and underscores. Once a field has been created, the identifier can not be changed.

Unit

Optional

Role

None

Assign roles to highlight key measurement fields across the platform; each role is limited to one field per product.

Semantic

None

Assign semantic fields to standardize data across devices.

Formula

Optional

Formulas can be used to perform calculations on values from other fields. Fields that have a formula can not be written to from a decoder or via the API.

☐ Use Formula

Cancel

Add Field

Add Field

Fields define the schema of the data the device stores.

Type

Boolean

Name

Door\_open\_flag

Identifier

DOOR\_OPEN\_FLAG

The field identifier is a unique string that can consist of uppercase letters, numbers and underscores. Once a field has been created, the identifier can not be changed.

Unit

Optional

Role

None

Assign roles to highlight key measurement fields across the platform; each role is limited to one field per product.

Semantic

None

Assign semantic fields to standardize data across devices.

Formula

Optional

Formulas can be used to perform calculations on values from other fields. Fields that have a formula can not be written to from a decoder or via the API.

☐ Use Formula

Cancel

Add Field

Add Field

Fields define the schema of the data the device stores.

Type

Numeric

Name

Door\_open\_cnt

Identifier

DOOR\_OPEN\_CNT

The field identifier is a unique string that can consist of uppercase letters, numbers and underscores. Once a field has been created, the identifier can not be changed.

Unit

Optional

Role

None

Assign roles to highlight key measurement fields across the platform; each role is limited to one field per product.

Semantic

None

Assign semantic fields to standardize data across devices.

Formula

Optional

Formulas can be used to perform calculations on values from other fields. Fields that have a formula can not be written to from a decoder or via the API.

☐ Use Formula

Cancel

Add Field

Add Field

Fields define the schema of the data the device stores.

Type

Numeric

Name

Door\_open\_last\_time

Identifier

DOOR\_OPEN\_LAST\_TIME

The field identifier is a unique string that can consist of uppercase letters, numbers and underscores. Once a field has been created, the identifier can not be changed.

Unit

Optional

Role

None

Assign roles to highlight key measurement fields across the platform; each role is limited to one field per product.

Semantic

None

Assign semantic fields to standardize data across devices.

Formula

Optional

Formulas can be used to perform calculations on values from other fields. Fields that have a formula can not be written to from a decoder or via the API.

☐ Use Formula

Cancel

Add Field

21

### Add Field

Fields define the schema of the data the device stores.

Type

Name

Identifier

The field identifier is a unique string that can consist of uppercase letters, numbers and underscores. Once a field has been created, the identifier can not be changed.

Unit

Role

Assign roles to highlight key measurement fields across the platform; each role is limited to one field per product.

Semantic

Assign semantic fields to standardize data across devices.

Formula

Formulas can be used to perform calculations on values from other fields. Fields that have a formula can not be written to from a decoder or via the API.

☐ Use Formula

Cancel Add Field

11、Press RST button, wait until the sensor connects to the gateway successfully, you will see the data the sensor is currently reading.

### Fields

Fields describe the data the device will store.

+ Add Field

● Live data

| NAME                | IDENTIFIER          | TYPE    | ROLE / SEMANTIC | CURRENT VALUE | LAST UPDATE   |
|---------------------|---------------------|---------|-----------------|---------------|---------------|
| Door_open_flag      | DOOR_OPEN_FLAG      | Boolean |                 | False         | 9 minutes ago |
| Door_open_cnt       | DOOR_OPEN_CNT       | Numeric |                 | 0             | 7 minutes ago |
| Door_open_last_time | DOOR_OPEN_LAST_TIME | Numeric |                 | 0             | 6 minutes ago |
| Door_open_all_time  | DOOR_OPEN_ALL_TIME  | Numeric |                 | 0             | 5 minutes ago |
| bat                 | BAT                 | Numeric |                 | 0             | in 0 seconds  |

12、To get a better look at the data, we can add widget.

Click “Dashboard-->switch-->+ Add Widget”.

**DATA CAKE**

Fleet > agrosense sensor

## agrosense sensor

Serial Number: 48E663FFFE30001F    Last update: Never

Dashboard

History

Downlinks

Configuration

Debug

Rules

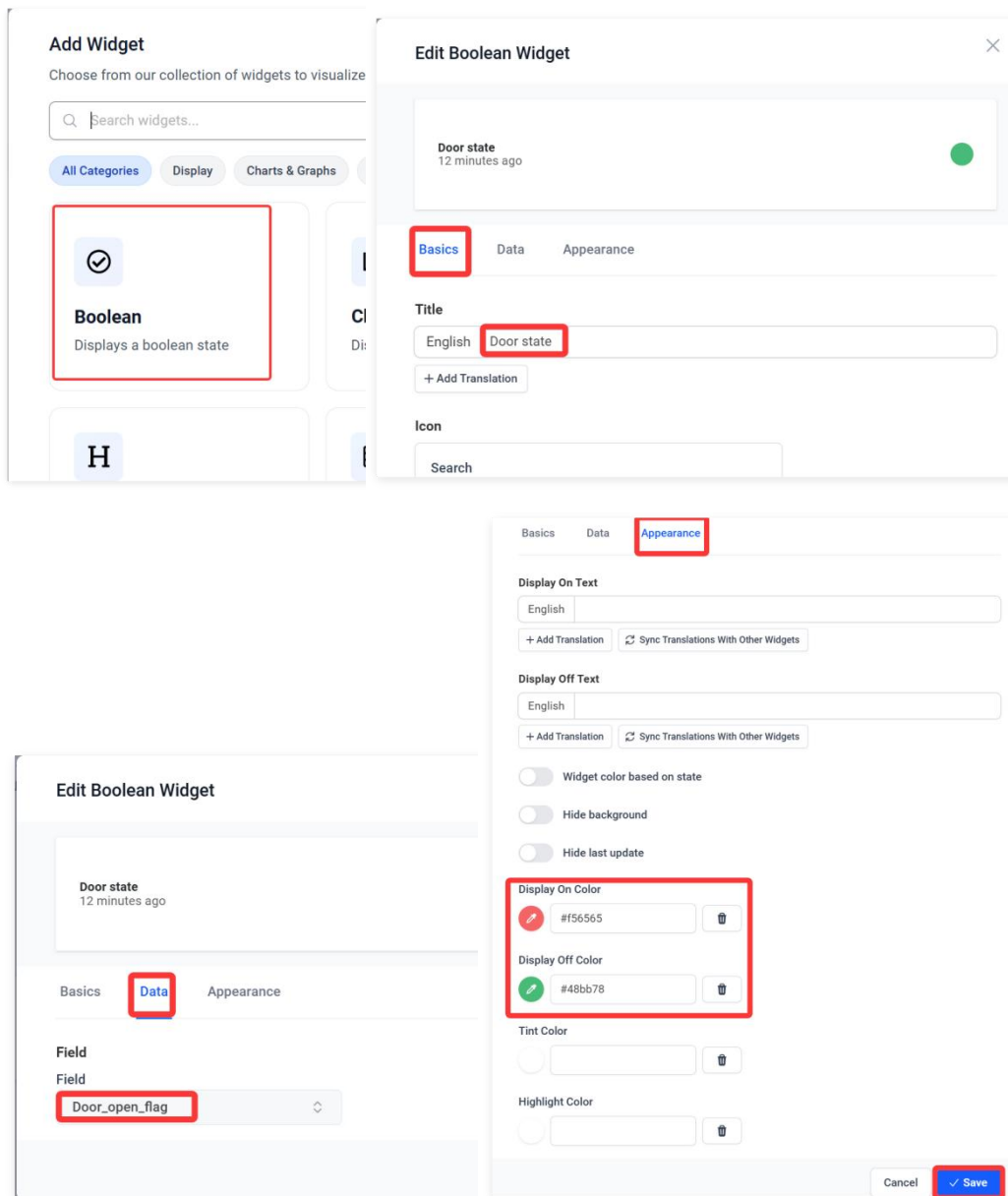
Permissions

Public Link

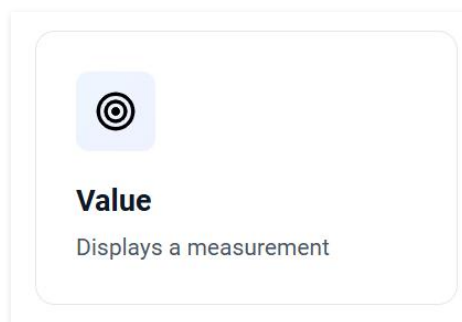
Add Widget

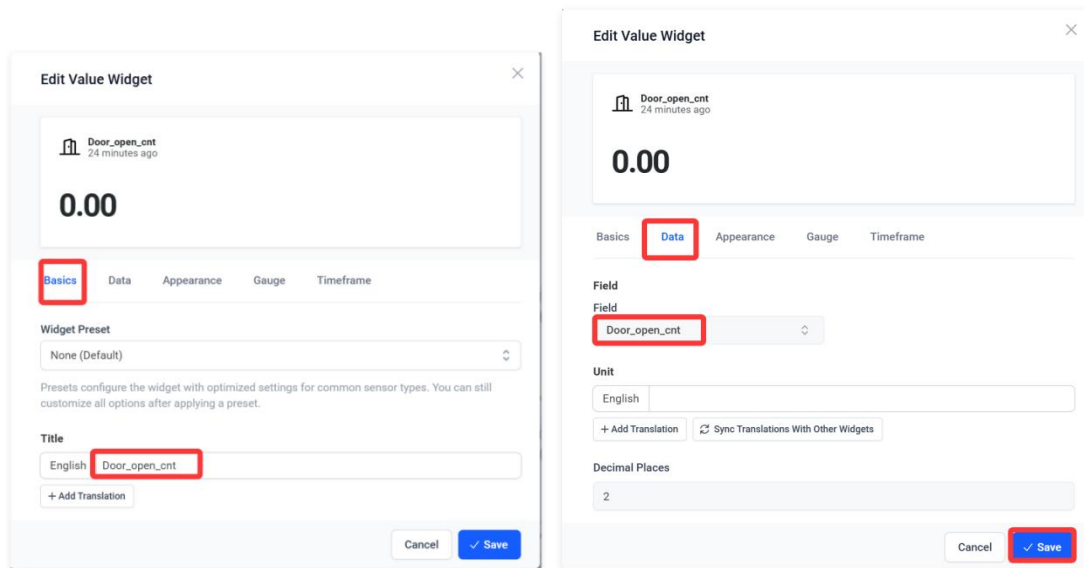


13、Select “Boolean” and set Title, Field as well as the state color.

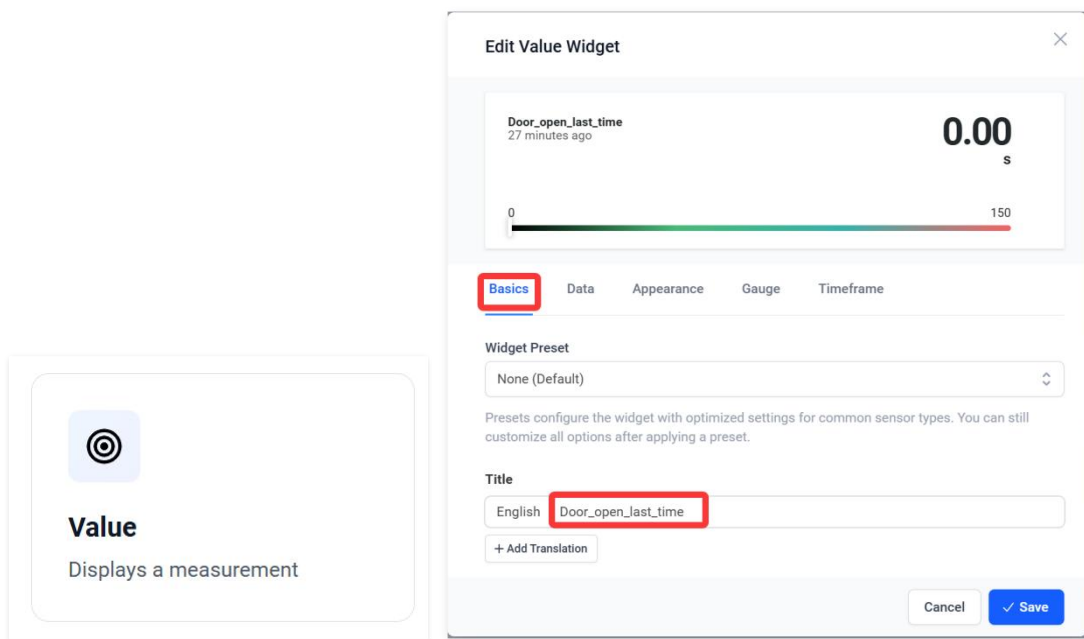


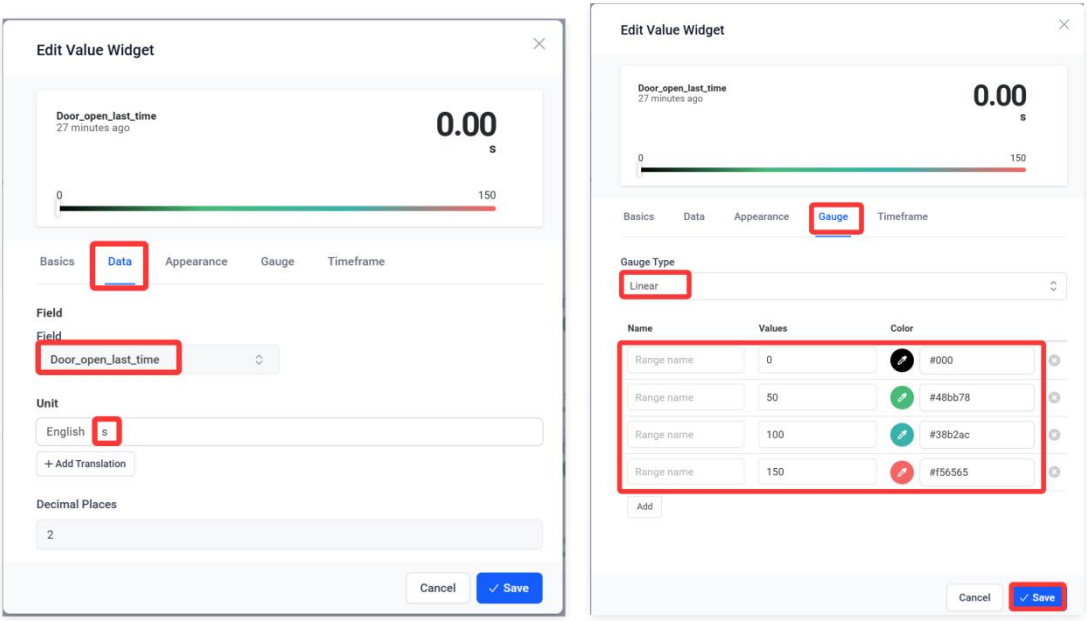
14、Select “Value” and set Title, Field and click “save”.



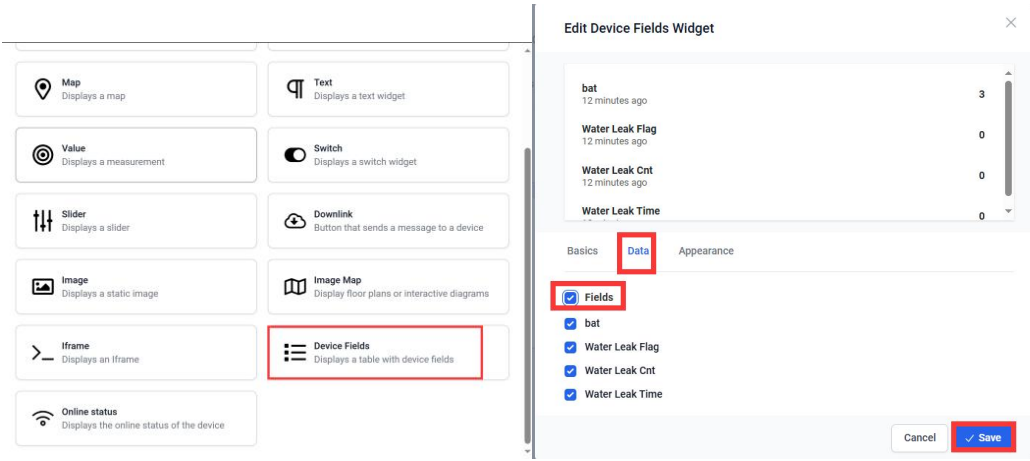


15、Select “Value” and set Title, Field and presentation form as well as the interval color. Door\_open\_all\_time as the same way.

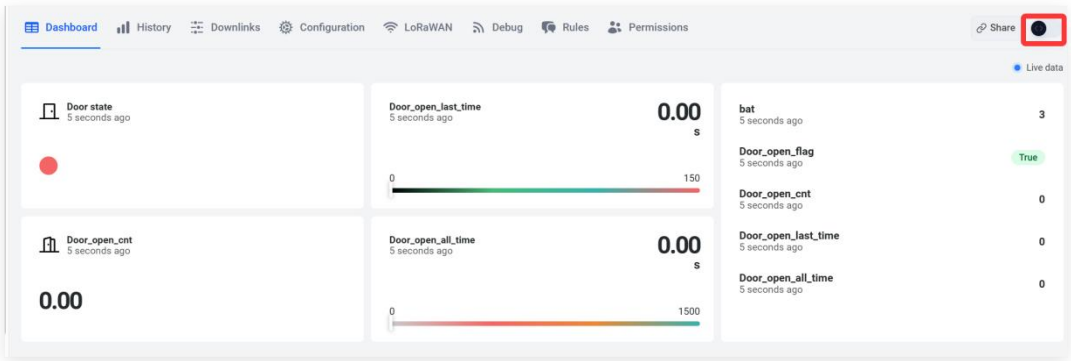




16、Select Device Fields, check “Fields” and click “Save”.



16、Click the switch to save, and you can see the data visually.



| Leakage situation          | Reporting of data                     |
|----------------------------|---------------------------------------|
| Normal state (Door close ) | Data reported every 12 hours.         |
| Occasional open            | Report once when opening is detected, |

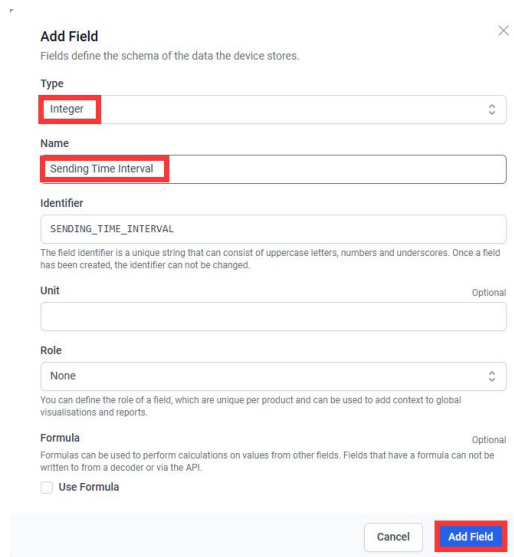
|                 |                                                      |
|-----------------|------------------------------------------------------|
|                 | and report again when closing is detected.           |
| Persistent open | Reported every 10 minute, up to 5 consecutive times. |

### 3.2.1 Downlink

- Modification time interval (Fport1)

Modify the time interval for uploading data, the default is one hour.

1 、 If you need to change time Interval (Default 60 minutes), you can click “Configuration-->Fields-->+Add Field”



**Add Field**

Fields define the schema of the data the device stores.

Type: Integer

Name: Sending Time Interval

Identifier: SENDING\_TIME\_INTERVAL

The field identifier is a unique string that can consist of uppercase letters, numbers and underscores. Once a field has been created, the identifier can not be changed.

Unit: Optional

Role: None

You can define the role of a field, which are unique per product and can be used to add context to global visualisations and reports.

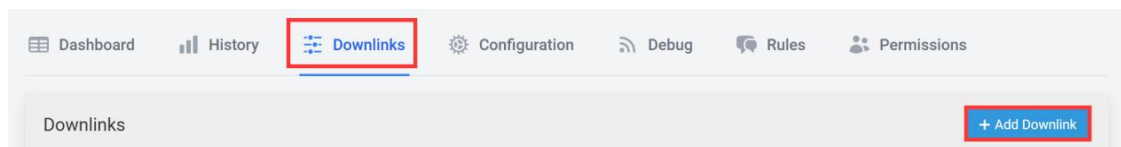
Formula: Optional

Formulas can be used to perform calculations on values from other fields. Fields that have a formula can not be written to from a decoder or via the API.

☐ Use Formula

Cancel Add Field

2、 Click “Downlink-->Add Downlink”.



Enter name、 description、 fields used and payload encoder respectively.

Name: Set User-Defined Sending Time Interval

Description: Set the user-defined report transmission interval and store it in the configuration variable.(5Min-1440Min)

Payload Encoder: copy in [Github](#).

**Configure Downlink**

**Name**  
Set User-Defined Sending Time Interval

**Description**  
Set the user-defined report transmission interval and store it in the configuration variable.(5Min-1440Min)

**Fields used**  
If your encoder function takes input from the device's fields, you can specify them here. They will be used to create the form for the downlink generator.  
SENDING\_TIME\_INTERVAL

☐ Trigger on measurements  
If activated, each time the device records a measurement in one of the fields used, the downlink will be sent automatically.

**Port**  
1

**Payload Encoder**

```

1 function Encoder(measurements, port) {
2   var interval = measurements["SENDING_TIME_INTERVAL"].value * 60;
3   if (interval < 300) {
4     interval = 300;
5     console.log("Interval < 300 Seconds / 5 Minutes not allowed!");
6   }
7   // Convert to hexadecimal only from interval
8   return interval.toString(16).padStart(4, '0').match(/.{2}/g).map(function(f) { return parseInt(f, 16);
9 });
10
11 // String.prototype.padStart() polyfill
12 // https://github.com/uxitten/polyfill/blob/master/string.polyfill.js
13 // https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/String/padStart
14 //
15 if (!String.prototype.padStart) {
16   String.prototype.padStart = function padStart(targetLength,padString) {
17     targetLength = targetLength>0; //truncate if number or convert non-number to 0;
18     padString = String(typeof padString !== 'undefined' ? padString : '');
19     if (this.length > targetLength) {
20       return String(this);
21     }
22     else {
23       targetLength = targetLength-this.length;
24       if (targetLength > padString.length) {
25         padString = padString.repeat(targetLength/padString.length); //append to original to en
26       }
27     }
28   }
29 }

```

3、Click “Dashboard-->switch-->+ Add Widget”.

Select “Downlink” and setting as follow image.

**Edit Downlink Widget**

User-Defined Time Interval(5Min-1440Min)

**Basics** | Data | Appearance

**Title**  
English: User-Defined Time Interval(5Min-1440Min)  
German: [dropdown]  
+ Add Translation

Cancel | Save

**Edit Downlink Widget**

User-Defined Time Interval(5Min-1440Min)

**Basics** | **Data** | Appearance

**Downlink**  
Set User-Defined Sending Time Interval

**Additional Downlinks**  
+ Add

Cancel | Save

4、Click the switch to save, and you can click to change your time Interval.

User-Defined Time Interval(5Min-1440Min)

**Sending Time Interval**

1

Cancel | Save measurements and send downlink